

Full Stack Developer - Angular & .NET Core Assignment

Objective:

Develop a 2-page web application based on the attached UI designs using Angular and ASP.NET Core Web API.

Your solution should demonstrate good software engineering practices, clean architecture, reusable components, responsive UI, and maintainable code.

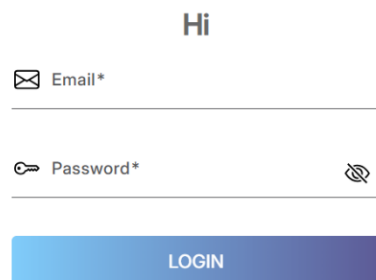
As part of this assignment, you are also required to:

- Implement a basic login function using Angular and C# backend.
- Create a simple Purchase Bill Add functionality, including form input, validation, and submission to the backend API.

Assignment Requirements

Task 1: Implement a Basic Login Page

Design and develop a simple login page based on the provided sample screenshot. The login functionality should authenticate the user using the API endpoint below:



The screenshot shows a login form with a blue header bar. Below the header, the text "Hi" is displayed. There are two input fields: "Email*" with an envelope icon and "Password*" with a key icon and a toggle eye icon. Below the password field is a blue "LOGIN" button.

API Endpoint:

https://ez-staging-api.azurewebsites.net/api/External_Api/POS_Api/Invoke

Use the sample request below by replacing the values entered by the user.

`</>` JSON

```
{
  "API_Action": "GetLoginData",
  "Device_Id": "D001",
  "Sync_Time": "",
  "Company_Code": "info@enhancer.com",
  "API_Body": {
    "Username": "info@enhancer.com",
    "Pw": "Welcome#5"
  }
}
```

- Replace "Company_Code" and "Username" with the email field from the form.
- Replace "Pw" with the value from the password field.

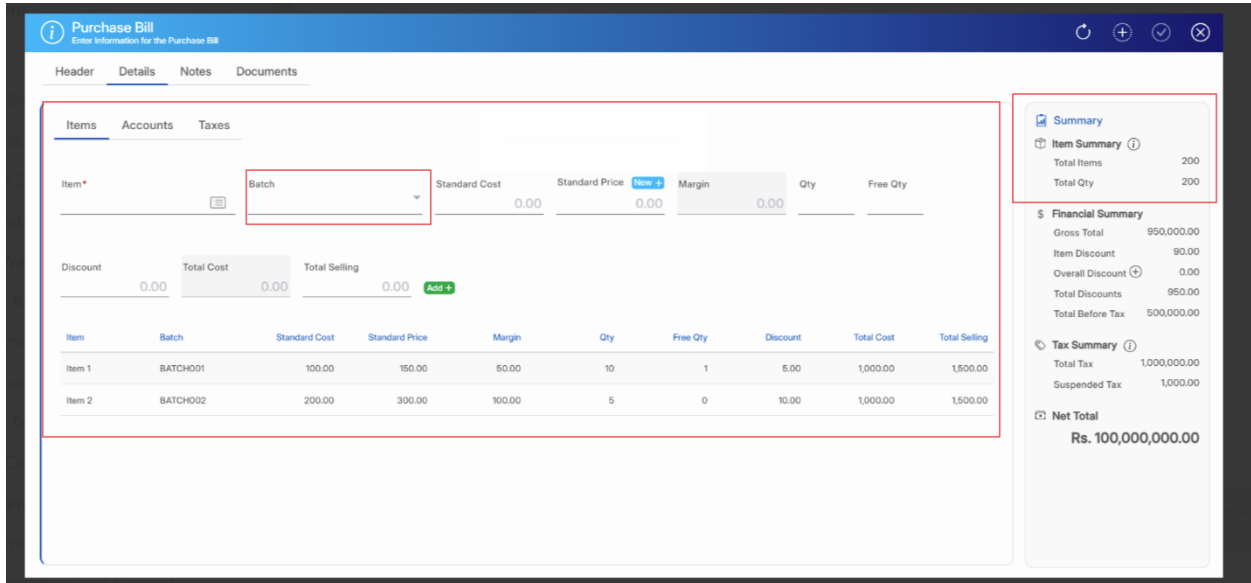
Login Logic:

- Validate user input.
- Prevent access if authentication fails.
- Display meaningful validation and error messages.
- Securely store the authenticated session.
- Protect the Purchase Bill page from unauthorized access.
- Upon successful login, the response contains a **User_Locations** collection.
- If the login is successful, the response will contain an array called User_Locations, with fields:
 - Location_Code
 - Location_Name

On successful login, Create a SQL Server database and save the locations in a table named Location_Details

Task 2: Create a Purchase Bill Form (Accessible After Login)

Develop a simple Purchase Bill form that is only accessible after a successful login (Task 1).



Purchase Bill
Enter Information for the Purchase Bill

Header Details Notes Documents

Items Accounts Taxes

Item* Batch Standard Cost Standard Price **New** Margin Qty Free Qty

Discount Total Cost Total Selling

Item Batch Standard Cost Standard Price Margin Qty Free Qty Discount Total Cost Total Selling

Item 1 BATCH001 100.00 150.00 50.00 10 1 5.00 1,000.00 1,500.00

Item 2 BATCH002 200.00 300.00 100.00 5 0 10.00 1,000.00 1,500.00

Summary

Item Summary

Total Items 200

Total Qty 200

Financial Summary

Gross Total 950,000.00

Item Discount 90.00

Overall Discount 0.00

Total Discounts 950.00

Total Before Tax 500,000.00

Tax Summary

Total Tax 1,000,000.00

Suspended Tax 1,000.00

Net Total

Rs. 100,000,000.00

Form Details:

- Item Field: Autocomplete with the following values:
"Mango", "Apple", "Banana", "Orange", "Grapes", "Kiwi", "Strawberry"
- Batch Dropdown: Populate with the Location_Name values from the previously saved Location_Details table.

Field Calculations:

For example, given:

- Standard Cost = 100
- Standard Price = 150
- Quantity = 5
- Discount = 20%

Then:

- Total Cost = (Standard Cost × Quantity) – Discount% = $100 \times 5 - (20\%) = 400$
- Total Selling = Standard Price × Quantity = $150 \times 5 = 750$

After clicking “Add”:

- The item should be added to a table displayed below.
- The Item Summary Section should update with:
 - Total Items = Number of rows in the table
 - Total Quantity = Sum of all Qty fields in the table

Technologies:

Use

- Angular (Latest)
- TypeScript
- ASP.NET Core (.NET 8 preferred)
- C#
- SQL Server

The application should demonstrate

- Component-based architecture
- Reusable Angular components
- Strong typing
- REST API integration
- Form validation
- Responsive UI
- Clean code
- Proper folder structure
- Error handling
- Loading indicators
- Meaningful commit history

Submission:

Please submit the following:

- GitHub Repository URL
- SQL Server database script (.sql)
- Project README with setup instructions
- 5–10 minute screen recording demonstrating your solution
- Completed submission form

Important

AI-assisted development (ChatGPT, GitHub Copilot, Claude, etc.) is permitted. However, candidates must fully understand their implementation and should be able to explain any part of their solution during the technical interview.

Contact Information:

If you have any questions, please contact on careers@enhancer.com.

(please reply to this email thread)